

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

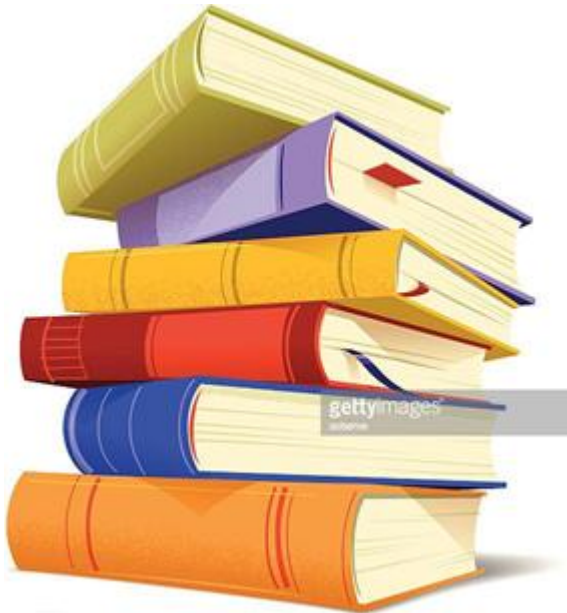
www.hoasen.edu.vn

Chương 3 – Phần 2

Các thành phần toán học nền tảng của mạng nơ-ron



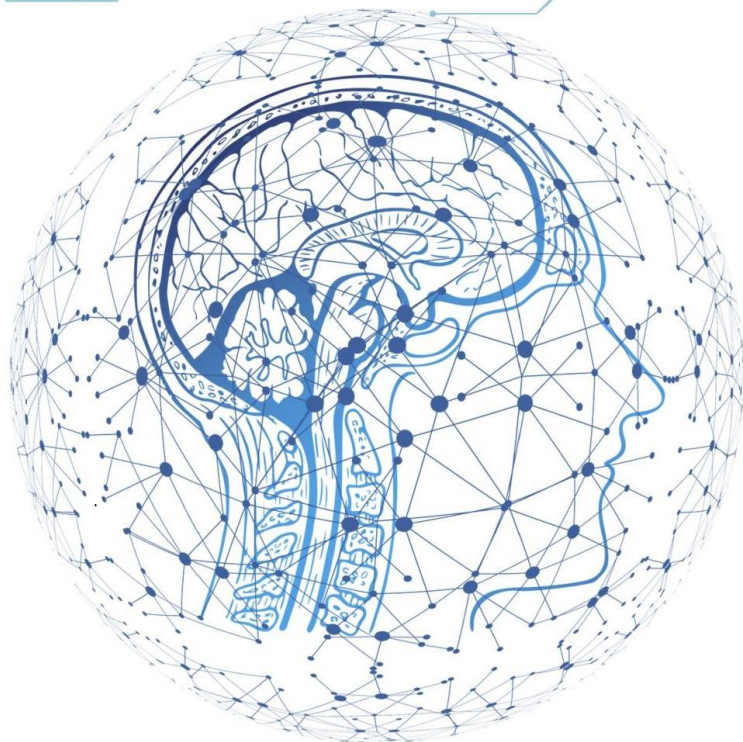
Các thành phần toán học nền tảng của mạng nơ-ron



- Ví dụ đầu tiên về mạng nơ-ron
- Tensor và các phép toán tensor
- Cách mạng nơ-ron học thông qua thuật toán Backpropagation & Gradient Descent

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Cái nhìn đầu tiên về một mạng nơ-ron

Bài toán phân loại chữ viết tay

■ Bài toán

- Phân loại ảnh chữ số viết tay 28×28 pixel
- Có 10 lớp: từ 0 đến 9
- Bộ dữ liệu sử dụng: **MNIST**

■ MNIST gồm

- 60.000 ảnh huấn luyện
- 10.000 ảnh kiểm tra
- Ảnh grayscale, kích thước nhỏ, kinh điển trong deep learning

■ Khái niệm cần nhớ

- **Class:** lớp/loại
- **Sample:** mẫu dữ liệu
- **Label:** nhãn của mẫu



Dữ liệu đầu vào trong ví dụ MNIST

■ Dữ liệu huấn luyện

- `train_images.shape = (60000, 28, 28)`
- `train_labels.shape = (60000,)`

■ Dữ liệu kiểm tra

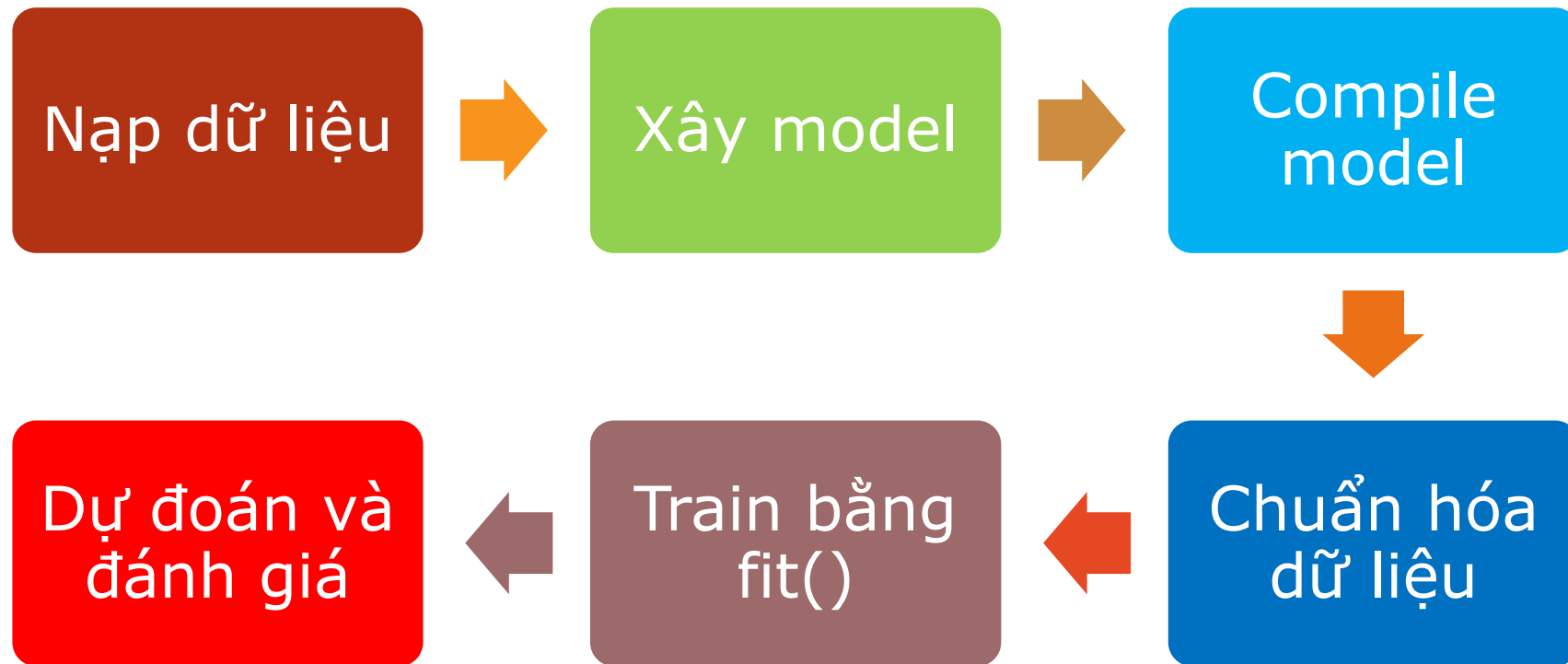
- `test_images.shape = (10000, 28, 28)`
- `test_labels.shape = (10000,)`

■ Ý nghĩa

- 60.000 ảnh để học
- 10.000 ảnh chưa thấy trước để đánh giá
- Mỗi ảnh tương ứng đúng một nhãn



Quy trình chung



Kiến trúc mạng nơ-ron đầu tiên

■ Kiến trúc

- 1 lớp Dense có 512 neuron, activation = ReLU
- 1 lớp Dense có 10 neuron, activation = Softmax

■ Ý nghĩa

- Lớp 1: học biểu diễn trung gian hữu ích
- Lớp 2: trả về xác suất cho 10 lớp

■ Softmax

- Đầu ra là 10 xác suất
- Tổng các xác suất bằng 1
- Xác suất cao nhất là lớp dự đoán

Compile, preprocess và train

■ Biên dịch model

- optimizer = rmsprop
- loss = sparse_categorical_crossentropy
- metrics = accuracy

■ Tiền xử lý

- Reshape từ (60000, 28, 28) thành (60000, 784)
- Chuyển kiểu sang float32
- Chia cho 255 để đưa dữ liệu về khoảng [0, 1]

■ Huấn luyện

- epochs = 5
- batch_size = 128

■ Ghi nhớ

- Optimizer quyết định cách cập nhật trọng số.
- Loss đo sai số của model.
- Metrics là các chỉ số để theo dõi hiệu quả của model trong quá trình train và test. Accuracy = tỷ lệ dự đoán đúng.
- Preprocess giúp dữ liệu phù hợp với cấu trúc model → giúp model học ổn định hơn.

Kết quả dự đoán và hiện tượng overfitting

▪ Sau huấn luyện

- Model có thể dự đoán xác suất cho từng ảnh mới
- Chọn lớp có xác suất lớn nhất làm kết quả

Vì sao accuracy trên test lại thấp hơn trên train?

▪ Kết quả điển hình

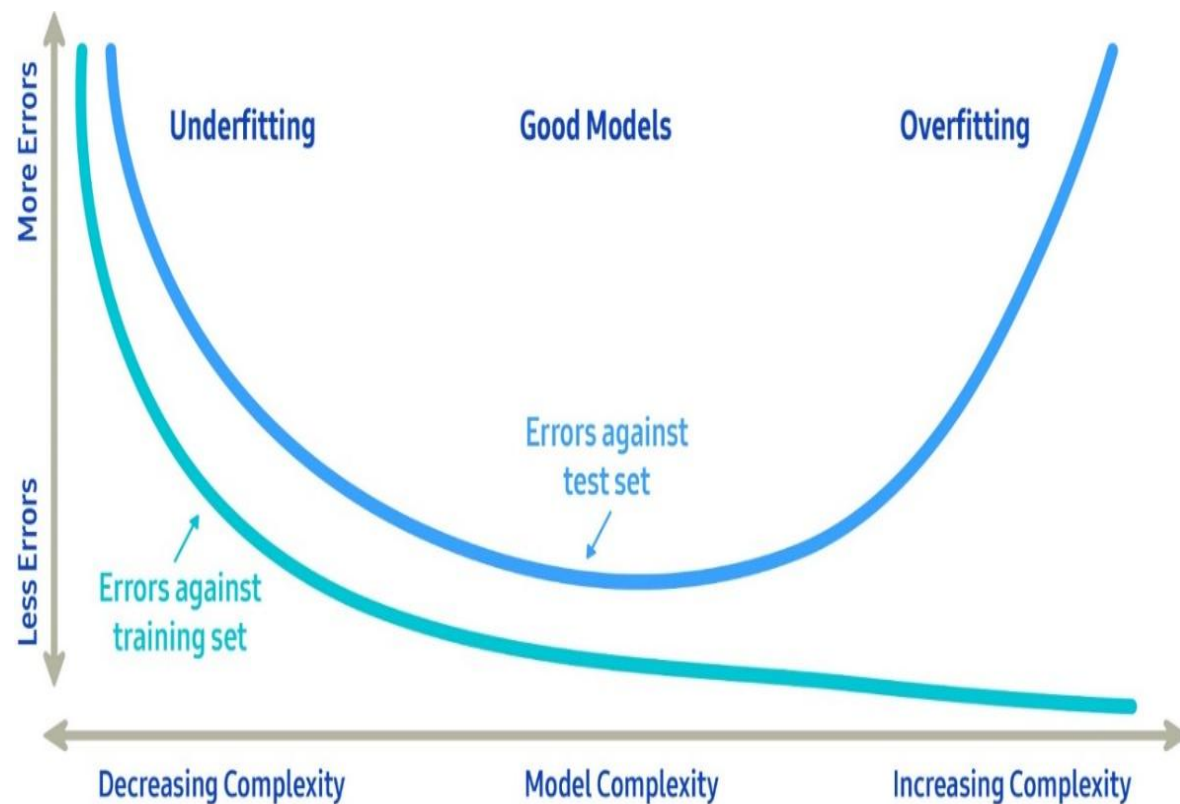
- Accuracy trên training cao hơn test
- Ví dụ trong sách: test accuracy khoảng **97.8%**

▪ Thông điệp quan trọng

- Model học tốt trên training chưa chắc tốt trên dữ liệu mới
- Khoảng cách giữa training và test là dấu hiệu của **overfitting**

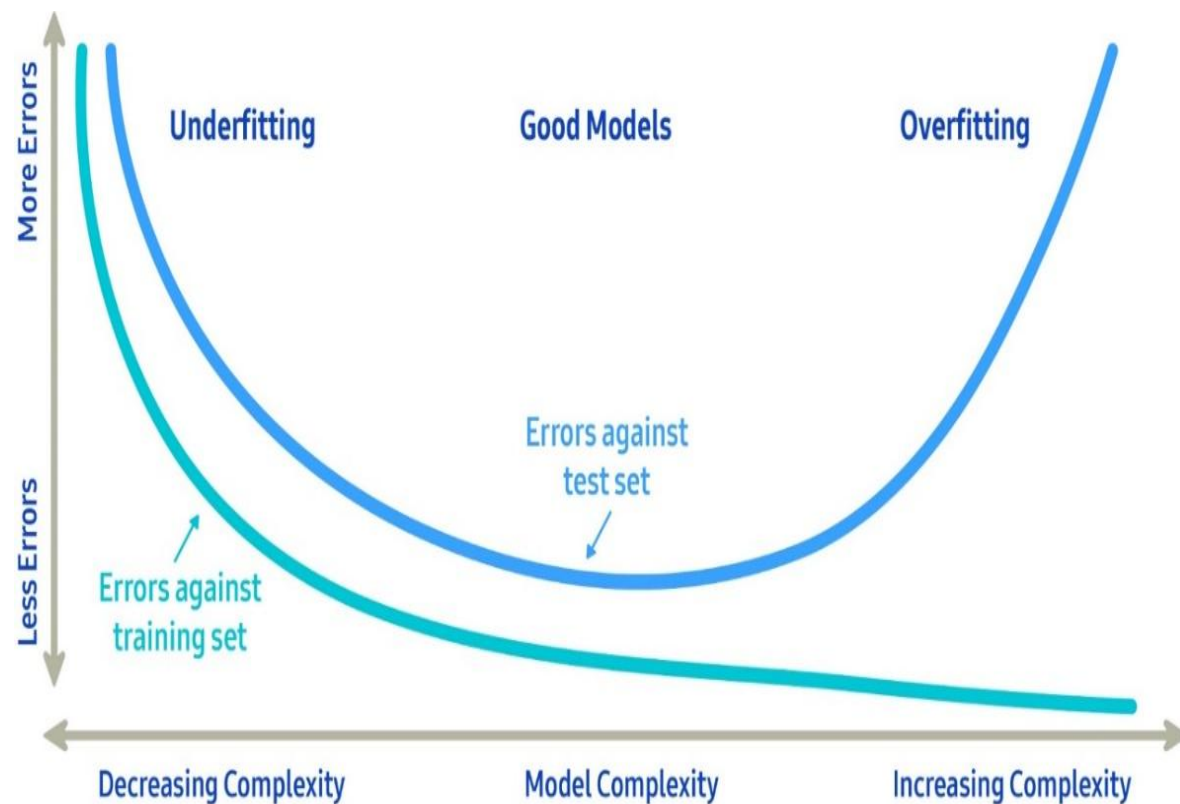
Overfitting và Underfitting

- **Overfitting (quá khớp):** mô hình học quá kỹ dữ liệu huấn luyện, kể cả nhiễu, nên **khó tổng quát hóa trên dữ liệu mới**.
- Hiệu suất rất tốt trên **train**
- Hiệu suất kém trên **test**
- Mô hình “học thuộc” thay vì “học hiểu”



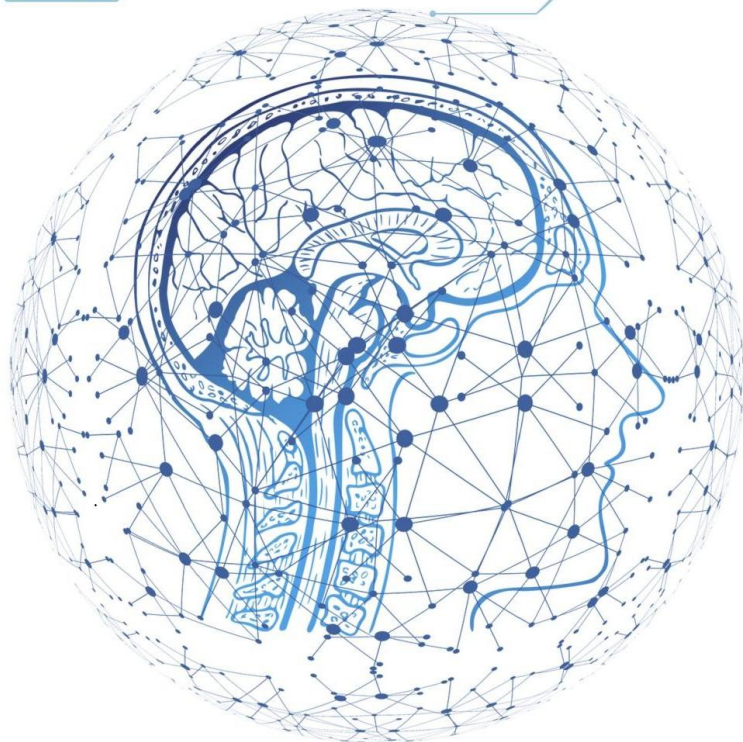
Overfitting và Underfitting

- **Underfitting (dưới khớp):** mô hình quá đơn giản nên **không học được quy luật của dữ liệu**.
- Hiệu suất kém trên **train**
- Hiệu suất cũng kém trên **test**
- Mô hình chưa nắm bắt được mẫu dữ liệu
- **Ví dụ:** Dùng đường thẳng để biểu diễn dữ liệu có dạng cong phức tạp.



DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Biểu diễn dữ liệu cho
mạng nơ-ron

Tensor

- Tensor là cấu trúc nền tảng của hầu hết hệ thống học máy hiện đại
- Tensor là sự mở rộng của ma trận sang nhiều chiều hơn
- **Từ khóa:**
 - **rank / ndim:** số chiều
 - **shape:** kích thước mỗi chiều
 - **dtype:** kiểu dữ liệu của phần tử
 - **axis:** trục đang thao tác
- **Ví dụ:**
 - $x = \begin{bmatrix} [1, 2], [3, 4], [5, 6] \\ [7, 8], [9, 10], [11, 12] \end{bmatrix}$
 - Tensor này có:
 - rank = 3
 - shape = (2, 3, 2)
 - dtype có thể là int32
 - axis=0: trục tương ứng với chiều ngoài cùng, axis=1: chiều giữa, axis=2: chiều trong cùng

Scalar, Vector, Matrix, Tensor

■ **Scalar (rank-0 tensor)**

- Chỉ chứa một số duy nhất
- VD: 3, 5, 12

■ **Vector (rank-1 tensor)**

- Một dãy số
- Có 1 trục
- VD: [2, 5, 9]

■ **Matrix (rank-2 tensor)**

- Bảng số 2 chiều
- Có hàng và cột
- VD: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

■ **Rank-3 trở lên**

- Tập hợp các matrix
- Rank-4, rank-5 dùng nhiều trong ảnh, video

Ba thuộc tính quan trọng của tensor

■ 1. Rank / Number of axes

- Số trục của tensor

■ 2. Shape

- Kích thước trên từng trục
- Ví dụ: (60000, 28, 28)

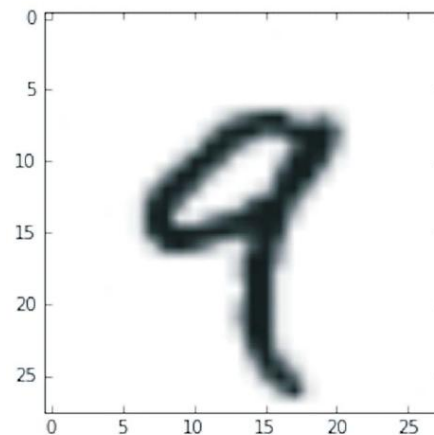
■ 3. Dtype

- Kiểu dữ liệu: uint8, float32, float64, ...

Mẫu thứ tư của dataset

■ Ví dụ từ MNIST

- `train_images.ndim = 3`
- `train_images.shape = (60000, 28, 28)`
- `train_images.dtype = uint8`



Tensor slicing và data batches

- **Tensor slicing**
- Lấy một phần tensor theo chỉ số
- Ví dụ chọn ảnh từ 10 đến 99
- Có thể cắt vùng con của ảnh

Các dạng tensor ngoài thực tế

■ 1. Vector data

- Shape: (samples, features)

■ 2. Timeseries / sequence data

- Shape: (samples, timesteps, features)

■ 3. Image data

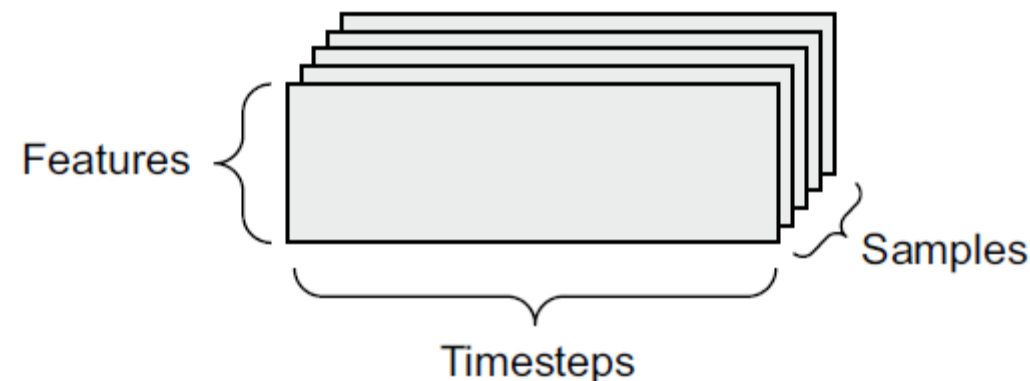
- Shape: (samples, height, width, channels)

■ 4. Video data

- Shape: (samples, frames, height, width, channels)

■ Ví dụ

- Bảng thông tin khách hàng → vector data
- Chuỗi giá cổ phiếu → timeseries
- Ảnh màu → rank-4 tensor
- Video → rank-5 tensor



Bộ truyền động của mạng nơ-ron: tensor operations

- **Trong lớp Dense**

$$output = relu(dot(input, W) + b)$$

- **Có 3 phép toán chính**

- Dot product
- Addition
- ReLU

- **Ý nghĩa**

- Layer thực chất chỉ là một chuỗi phép toán tensor
- Neural network = nhiều layer = nhiều phép toán tensor ghép lại

Element-wise operations và Broadcasting

■ **Element-wise operations**

- Cộng, trừ, nhân từng phần tử
- ReLU cũng là phép áp dụng riêng lẻ lên từng phần tử

■ **Broadcasting**

- Cho phép thao tác giữa tensor khác shape khi có thể mở rộng hợp lý
- Ví dụ: cộng một matrix với một vector

■ **Ý nghĩa**

- Giúp code ngắn gọn
- Tính toán hiệu quả, tận dụng vectorization

Dot product và reshape

■ Dot product

- Phép toán trọng tâm trong lớp Dense
- Kết hợp thông tin đầu vào với trọng số

■ Điều kiện shape

- Hai tensor phải tương thích chiều trong

■ Reshape

- Thay đổi hình dạng tensor mà không đổi số phần tử
- Ví dụ ảnh $28 \times 28 \rightarrow$ vector 784

■ Ứng dụng

- Chuẩn bị dữ liệu cho model
- Chuyển dữ liệu giữa các tầng có cấu trúc khác nhau

Diễn giải hình học của các phép toán tensor

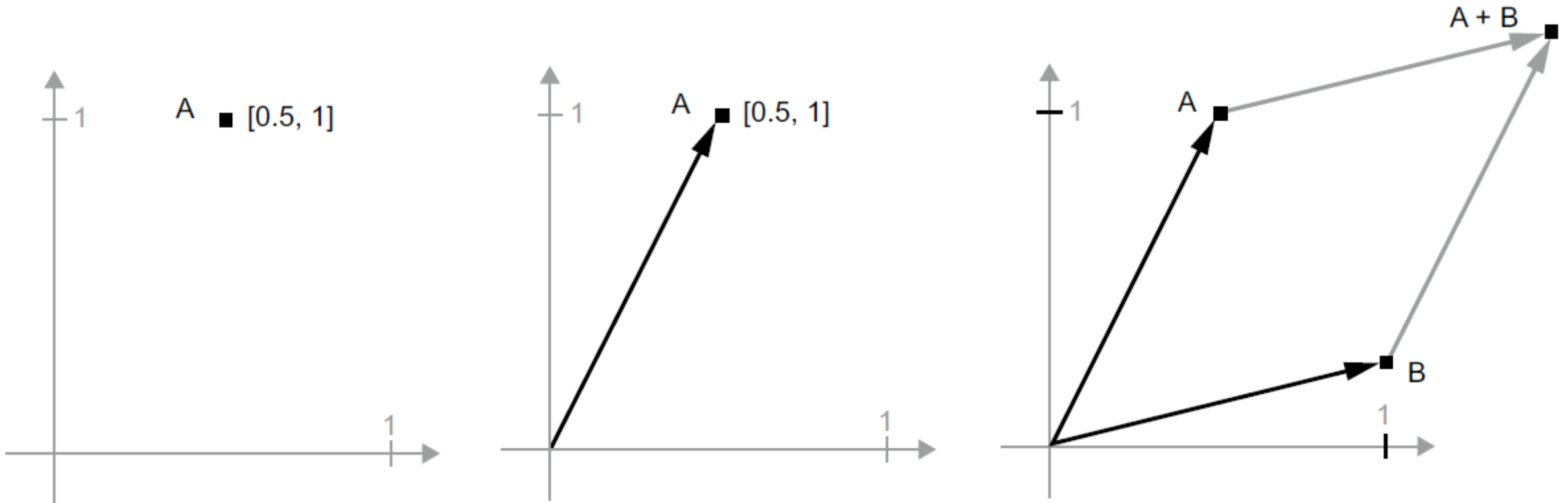
■ Các phép toán tensor có thể hiểu theo hình học

- Cộng vector $\rightarrow$ tịnh tiến
- Nhân ma trận $\rightarrow$ quay, co giãn, biến đổi tuyến tính
- Affine transform $\rightarrow$ biến đổi tuyến tính + tịnh tiến
- Dense layer không activation $\rightarrow$ chỉ là affine transform

■ Ý nghĩa rất quan trọng

- Nếu chỉ xếp nhiều lớp tuyến tính chồng lên nhau, mô hình vẫn chỉ là tuyến tính
- Cần activation như **ReLU** để tạo phi tuyến

Diễn giải hình học của các phép toán tensor



Một điểm trong 2D Space

Một điểm trong 2D Space
được biểu diễn như vector

Giải thích hình học
của tổng của hai vectơ

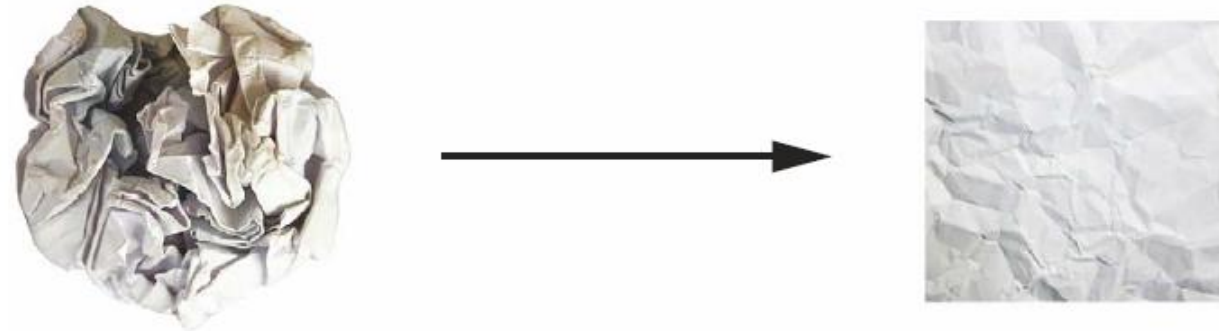
Diễn giải hình học của deep learning

■ Deep learning có thể hiểu là

- Một chuỗi dài các phép biến đổi hình học đơn giản
- Mỗi layer “gỡ rối” dữ liệu thêm một chút
- Sau nhiều lớp, dữ liệu trở nên dễ phân loại hơn

■ Thông điệp

- Deep learning mạnh vì chia một biến đổi rất phức tạp thành nhiều bước nhỏ



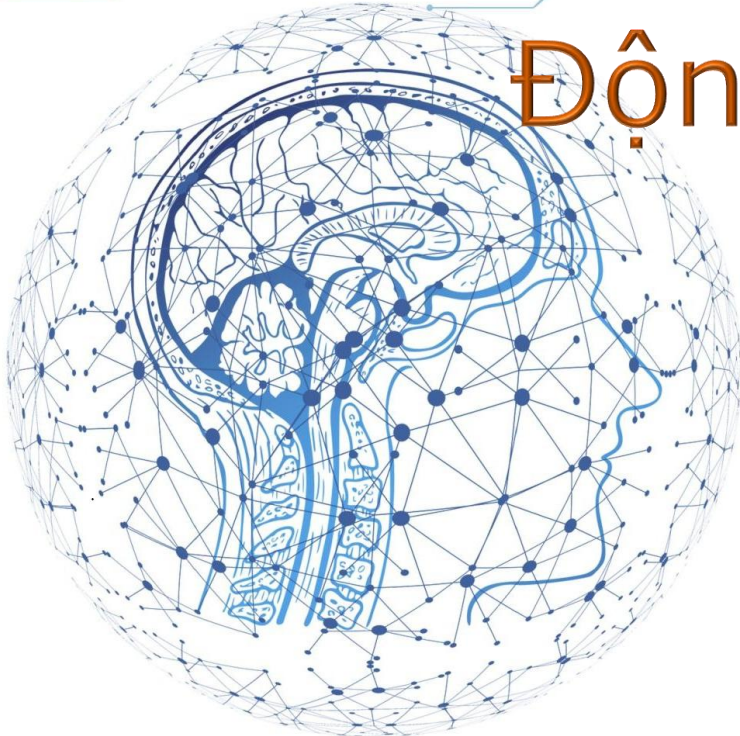
Ẩn dụ trong sách

- Hai tờ giấy màu bị vò nhàu vào nhau
- Mạng nơ-ron học cách “gỡ” chúng ra để tách biệt hai lớp dữ liệu

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Động cơ học của mạng nơ-ron:
tối ưu dựa trên gradient



Mạng học bằng cách nào?

■ Ý tưởng:

- Ban đầu trọng số là ngẫu nhiên
- Dự đoán sẽ sai
- Tính loss để đo độ sai
- Dùng gradient để biết nên chỉnh trọng số theo hướng nào
- Lặp lại nhiều lần

■ Training loop cơ bản

- Lấy một batch dữ liệu
- Forward pass
- Tính loss
- Tính gradient
- Cập nhật weights

Derivative, Gradient và Gradient Descent

■ Derivative

- Với hàm một biến: cho biết độ dốc tại một điểm

■ Gradient

- Mở rộng derivative cho nhiều biến/tensor
- Chỉ ra hướng tăng nhanh nhất của hàm loss

■ Gradient descent

- Muốn loss giảm thì đi ngược hướng gradient
- Công thức trực giác:
 - $W = W - \text{learning_rate} * \text{gradient}$

■ Learning rate

- Quá nhỏ → học chậm
- Quá lớn → dễ dao động, vượt quá điểm tối ưu

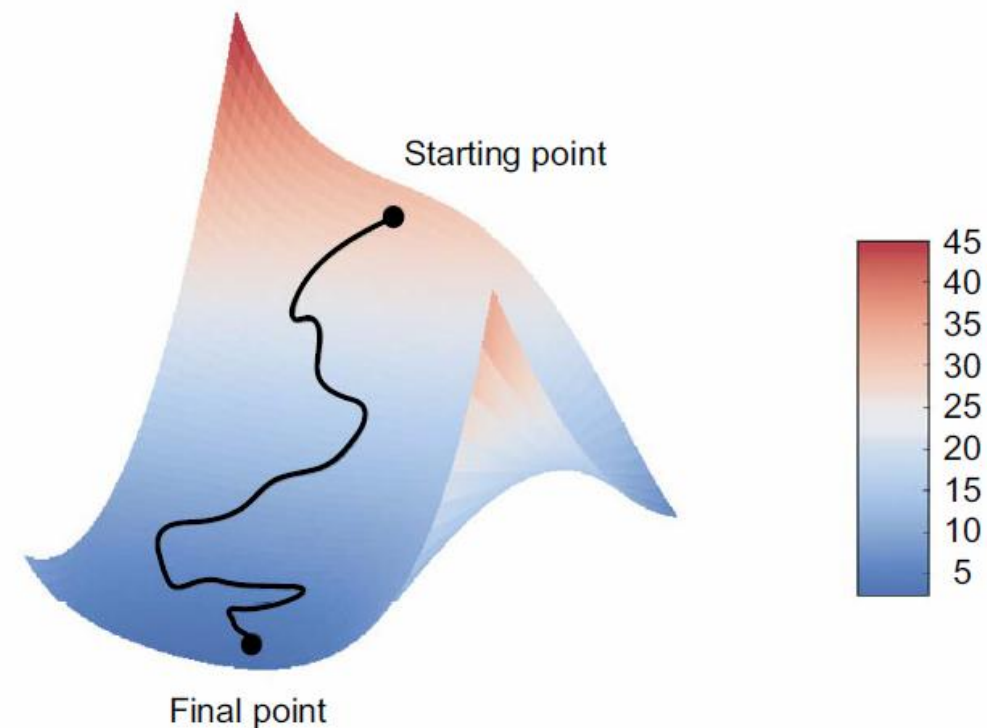
Biển thể Gradient Descent

■ Biển thể Gradient Descent

- Gradient Descent chuẩn (Batch Gradient Descent): dùng toàn bộ dataset cho mỗi lần cập nhật.
- Stochastic Gradient Descent (SGD): dùng 1 mẫu dữ liệu duy nhất cho mỗi lần cập nhật.
- Mini-batch Gradient Descent: dùng một batch nhỏ cho mỗi lần cập nhật.

■ Vì sao dùng mini-batch?

- Nhanh hơn
- Tiết kiệm tài nguyên
- Thực tế hơn batch gradient descent



Giảm độ dốc (Gradient Descent) xuống một bề mặt mất mát 2D (hai tham số có thể học được)

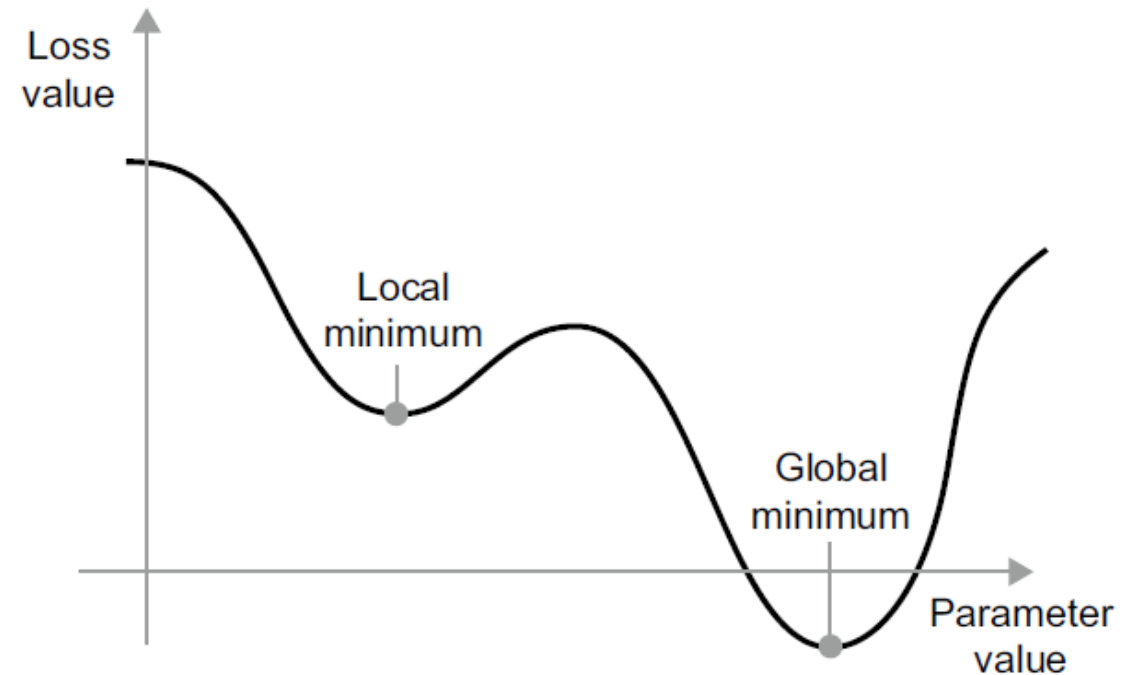
Momentum - phiên bản cải tiến của Gradient Descent

■ Momentum

- Thêm “quán tính” vào quá trình cập nhật
- Giúp đi nhanh hơn
- Giảm nguy cơ mắc kẹt ở local minimum nhỏ

■ Điểm khác biệt

- Gradient Descent chỉ “**nhìn hiện tại**”
- Momentum “**nhìn cả quá khứ + hiện tại**” → thông minh hơn trong việc chọn hướng đi



Cực tiểu cục bộ & toàn cục

Momentum

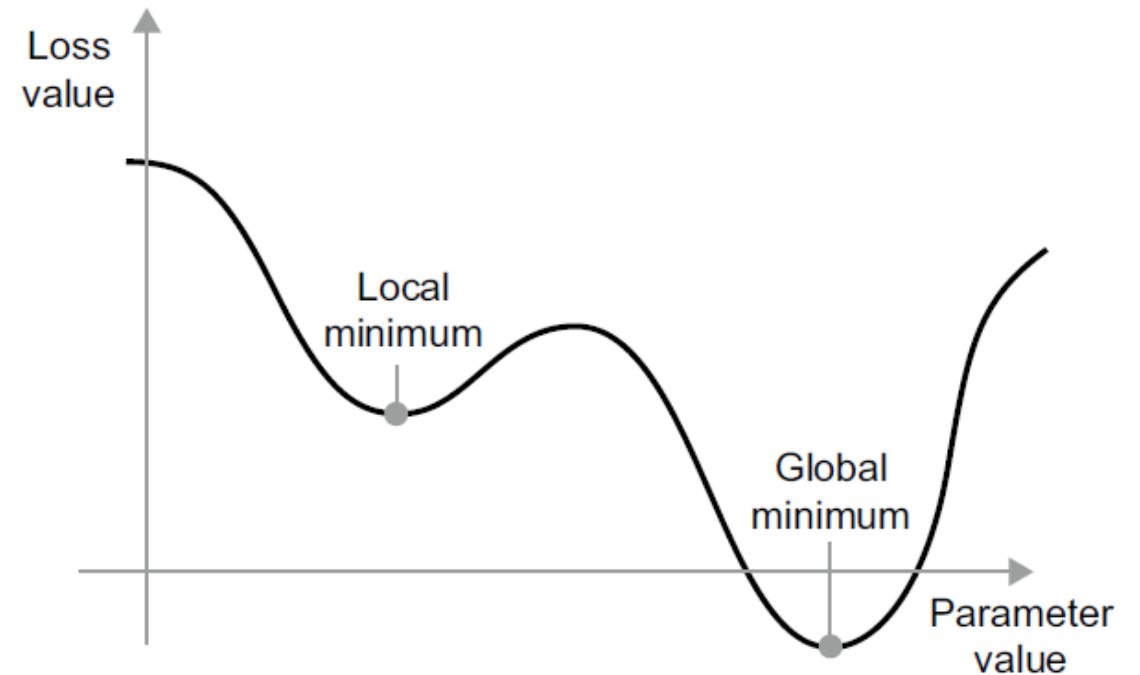
- Công thức cập nhật tham số:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla L$$

$$w = w - \alpha v_t$$

- Trong đó:

- v_t là vận tốc (momentum)
- β là hệ số quán tính (thường khoảng 0.9)
- ∇L là gradient
- w là tham số



Cực tiểu cục bộ & toàn cục

Backpropagation – Lan truyền ngược

■ Ý tưởng chính

- Neural network là chuỗi các phép toán đơn giản
- Mỗi phép toán có đạo hàm riêng
- Dùng **chain rule** để tính gradient của toàn bộ mạng

■ Backpropagation làm gì?

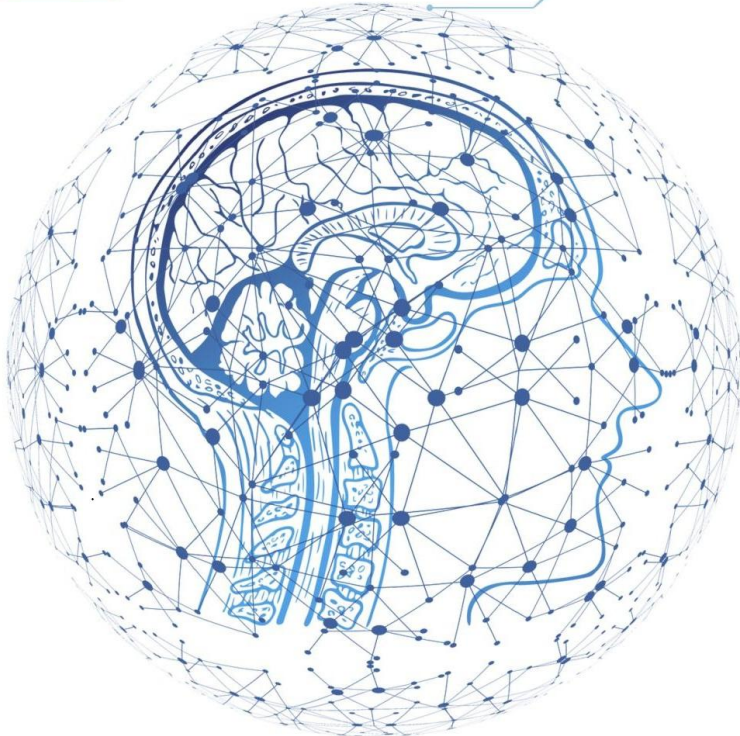
- Bắt đầu từ loss ở đầu ra
- Lan ngược trở lại qua từng lớp
- Tính mức đóng góp của từng tham số vào sai số

■ Vai trò

- Là nền tảng để cập nhật trọng số hiệu quả
- Không cần thử từng trọng số một cách ngây thơ

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Nhìn lại ví dụ đầu tiên

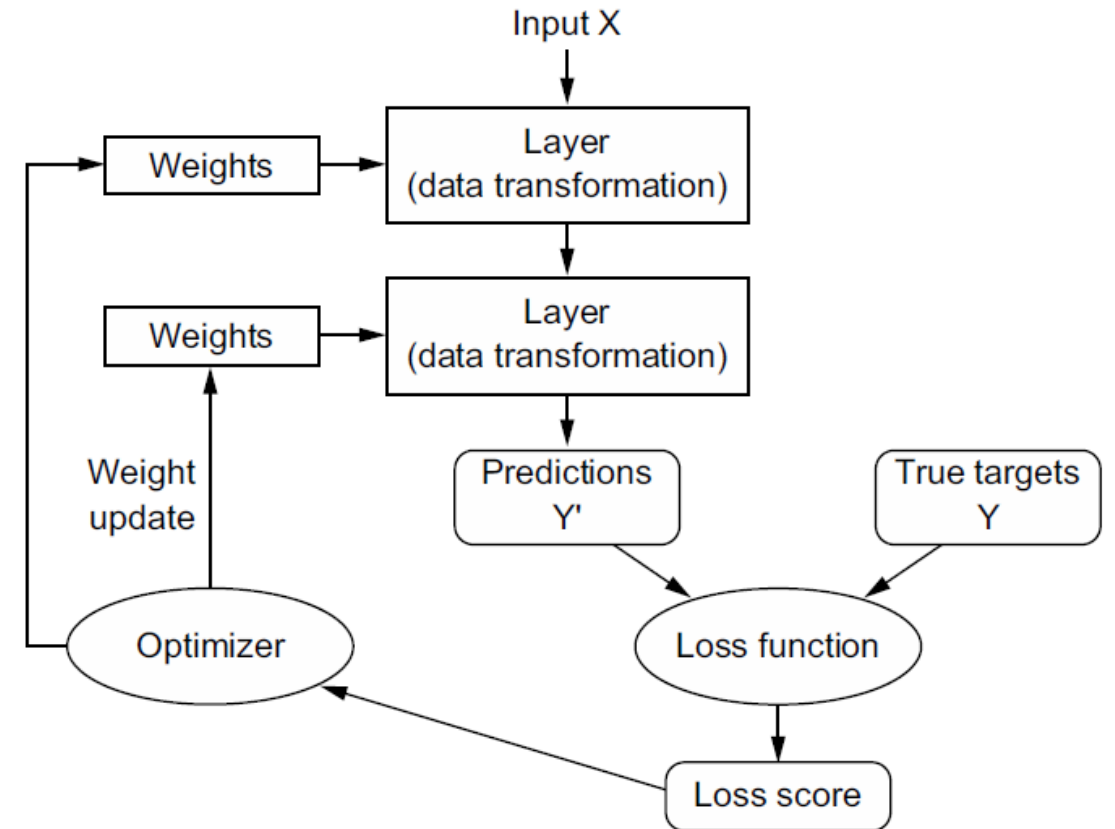
Nhìn ví dụ MNIST dưới góc nhìn toán học

■ Dưới góc nhìn toán học

- Input là tensor
- Layer là chuỗi phép toán tensor
- Weight là nơi lưu "tri thức" của model
- Loss đo độ sai
- Optimizer dùng gradient để cập nhật weight
- `fit()` thực hiện training loop lặp đi lặp lại

■ Training loop

- Data $\rightarrow$ Model $\rightarrow$ Predictions $\rightarrow$ Loss $\rightarrow$ Gradient $\rightarrow$ Update weights



Mối quan hệ giữa network, layers, loss function & optimizer

Tổng kết

- Tensor là cấu trúc dữ liệu trung tâm của deep learning
- Layer thực hiện các phép toán tensor
- Neural network là chuỗi các layer
- Học nghĩa tìm trọng số làm loss nhỏ nhất
- Gradient descent cập nhật trọng số theo hướng giảm loss
- Backpropagation là cơ chế tính gradient hiệu quả
- Loss và optimizer là hai khái niệm phải hiểu rất chắc

Câu hỏi thảo luận

1. Vì sao không thể nói neural network chỉ là “một đồng công thức phức tạp” mà nên hiểu nó như chuỗi biến đổi dữ liệu?
2. Tại sao cần activation function? Nếu bỏ activation thì chuyện gì xảy ra?
3. Sự khác nhau giữa derivative và gradient là gì?
4. Vì sao model thường train theo mini-batch thay vì toàn bộ dữ liệu?
5. Backpropagation giúp ta vượt qua hạn chế nào của cách cập nhật trọng số ngẫu nhiên?
6. Một file âm thanh hoặc một câu tiếng Việt nên biểu diễn giống vector data hay sequence data? Vì sao?

